

Domande teoria

1. **Definizione di tempo di calcolo di un algoritmo:** Il tempo di calcolo di un algoritmo rappresenta la quantità di tempo necessaria per eseguire l'algoritmo su un input specifico. È una misura dell'efficienza e delle prestazioni dell'algoritmo, influenzata dalla sua complessità, dalle dimensioni dell'input e dall'architettura del computer. Viene misurato in unità di tempo e può variare da algoritmo a algoritmo. La valutazione del tempo di calcolo è importante per scegliere l'algoritmo più adatto a un problema specifico.
2. **Tempo di calcolo di una procedura divide et impera:** Il tempo di calcolo di una procedura Divide et Impera varia a seconda dell'implementazione e della complessità del problema. In generale, si avvicina a $O(n \log n)$, ma può variare a seconda delle circostanze specifiche.
3. **Procedura generale divide et impera:** La procedura Divide et Impera si suddivide in tre fasi: dividere il problema originale in sottoproblemi più piccoli, risolvere i sottoproblemi in modo indipendente e combinarne le soluzioni per ottenere la soluzione finale. Questo approccio permette di affrontare problemi complessi in modo più gestibile, sfruttando la ricorsione e scomponendo il problema in porzioni più semplici da risolvere. La divisione riduce la complessità del problema iniziale, la soluzione dei sottoproblemi avviene in parallelo e infine le soluzioni parziali vengono combinate per ottenere il risultato desiderato.
4. **Un esempio a scelta di procedura divide et impera (Mergesort, Quicksort, MaxMin):** Prenderò l'esempio del Merge Sort, un algoritmo di ordinamento basato su Divide et Impera. Divide: L'array viene diviso a metà ricorsivamente finché ogni suddivisione contiene un singolo elemento o nessun elemento. Impera: I sottoproblemi vengono risolti in modo ricorsivo ordinando le suddivisioni dell'array separatamente utilizzando Merge Sort. Combina: Le suddivisioni ordinate vengono combinate insieme tramite l'operazione di "merge" per ottenere l'array ordinato finale. Ad esempio, prendendo l'array [5, 3, 8, 2, 1, 9, 4, 7], viene diviso in [5, 3, 8, 2] e [1, 9, 4, 7]. Queste suddivisioni vengono ulteriormente divise e successivamente combinate in modo ordinato fino ad ottenere l'array ordinato finale [1, 2, 3, 4, 5, 7, 8, 9]. Il Merge Sort utilizza il principio Divide et Impera per ordinare l'array in modo efficiente.
5. **Ricorsione: record di attivazione - saper simulare il fibonacci su un input:** Per simulare la sequenza di Fibonacci su un input utilizzando la ricorsione: Per simulare la sequenza di Fibonacci su un input utilizzando la ricorsione, definisci una funzione che prende un parametro intero "n". Se "n" è uguale a 0 o 1, restituisci direttamente "n". Altrimenti, richiama la stessa funzione con "n-1" e "n-2" come argomenti, somma i risultati delle chiamate ricorsive e restituisci il valore ottenuto. Ripeti questo processo fino a raggiungere i casi base. La ricorsione consente di calcolare i valori successivi basandosi sui valori precedenti fino a raggiungere il valore desiderato nella sequenza di Fibonacci.

6. **Quicksort:** Il Quicksort è un algoritmo di ordinamento basato su Divide et Impera. L'idea principale è selezionare un elemento chiamato "pivot" dall'array e suddividere gli elementi in due gruppi, uno con valori minori o uguali al pivot e l'altro con valori maggiori. Questa suddivisione viene applicata ricorsivamente su entrambi i gruppi fino a quando l'array risulta ordinato. Il tempo di calcolo del Quicksort in media è di $O(n \log n)$, ma nel caso peggiore può essere $O(n^2)$.
7. **Mergesort:** Il Mergesort è un altro algoritmo di ordinamento basato su Divide et Impera. L'approccio consiste nel suddividere l'array in due parti uguali, risolvere ricorsivamente i sottoproblemi su entrambe le parti e quindi combinare le soluzioni in modo ordinato. L'efficienza del Mergesort è garantita poiché richiede sempre $O(n \log n)$ operazioni di confronto e fusione degli elementi.
8. **Counting-sort:** Il Counting-sort è un algoritmo di ordinamento non basato su confronti, ma sfrutta le informazioni sulla distribuzione degli elementi nell'array. L'idea principale è contare il numero di occorrenze di ciascun valore nell'array e poi ricostruire l'array ordinato. È particolarmente efficiente per ordinare array di numeri interi non negativi con un range limitato. Il tempo di calcolo del Counting-sort è $O(n + k)$, dove "n" è la dimensione dell'array e "k" è il range dei valori.
9. **Heapsort:** Il Heapsort è un algoritmo di ordinamento basato su heap, una struttura dati di tipo albero binario speciale. L'approccio consiste nel costruire un heap max (o min) dall'array iniziale e quindi estrarre iterativamente l'elemento massimo (o minimo) dal heap per ottenere l'array ordinato. Il tempo di calcolo del Heapsort è sempre di $O(n \log n)$, indipendentemente dalla distribuzione dei dati.
10. **Confronto tra algoritmi:** La scelta tra questi algoritmi dipende dalle caratteristiche specifiche del problema, come la distribuzione dei dati, il range dei valori e la disponibilità di memoria aggiuntiva.
 - A. Quicksort ha una complessità media di $O(n \log n)$ ed è generalmente veloce, ma può avere un caso peggiore di $O(n^2)$ se la scelta del pivot non è bilanciata.
 - B. Mergesort ha una complessità sempre di $O(n \log n)$ ed è stabile, ma richiede spazio extra per la fase di combinazione delle soluzioni parziali.
 - C. Counting-sort ha una complessità di $O(n + k)$ ed è efficiente per array con un range di valori limitato, ma richiede memoria aggiuntiva per tenere conto delle occorrenze.
 - D. Heapsort ha sempre una complessità di $O(n \log n)$ ed è efficiente per grandi quantità di dati, ma richiede spazio extra per la struttura dati heap.
11. **In che cosa è diverso l'heapsort da quicksort?** Heapsort e Quicksort sono due algoritmi di ordinamento differenti. Il Quicksort utilizza confronti diretti e un pivot per suddividere l'array, mentre l'Heapsort sfrutta una struttura dati chiamata heap. L'Heapsort ha complessità costante di $O(n \log n)$, mentre il Quicksort ha un caso peggiore di $O(n^2)$. L'Heapsort è stabile, mentre il Quicksort no. L'Heapsort richiede più spazio per l'heap, mentre il Quicksort ne richiede meno. La scelta dipende dalle caratteristiche dei dati e dalle necessità specifiche del problema.

12. **Cos'è un heap?** Uno heap è una struttura dati che organizza un insieme di elementi in un albero binario completo o quasi completo, dove ogni nodo genitore ha un valore maggiore o minore rispetto ai suoi nodi figli. Può essere un heap max o min, con il valore massimo o minimo nella radice. Gli heap sono utilizzati per ottenere efficientemente il valore massimo o minimo e hanno applicazioni come l'ordinamento, le code di priorità e gli algoritmi di ricerca efficienti.
13. **Definizione induttiva di albero:** Un albero è una struttura dati costituita da un insieme di nodi interconnessi, in cui ogni nodo può avere zero o più nodi figli. L'albero ha una radice speciale che rappresenta il nodo di partenza dell'albero. Ogni nodo figlio è a sua volta la radice di un sottoalbero, formando una struttura gerarchica. L'induzione viene utilizzata per costruire l'albero partendo da un caso base di albero vuoto e aggiungendo progressivamente nodi e collegamenti. Questo processo induttivo garantisce che ogni albero sia un insieme ben definito di nodi e archi che rispetta la struttura gerarchica.
14. **Che tipo di input richiede la procedura Max-heapify e cosa restituisce (output)?** La procedura Max-Heapify richiede un array e un indice del nodo da "ripulire" in un heap max. Ripristina la proprietà dell'heap, scambiando l'elemento più grande tra il nodo padre e i suoi figli. Questo processo si applica ricorsivamente ai sottoalberi interessati. L'input è un array e un indice di un nodo. Non restituisce un output esplicito, ma modifica l'array per rispettare la proprietà dell'heap.
15. **Definizione albero binario di ricerca:** Un albero binario di ricerca è una struttura dati in cui ogni nodo ha al massimo due figli: un figlio sinistro e un figlio destro. Inoltre, per ogni nodo, tutti i valori presenti nel sottoalbero sinistro sono inferiori o uguali al valore del nodo, mentre tutti i valori nel sottoalbero destro sono maggiori o uguali al valore del nodo. Questa proprietà è chiamata proprietà di ordinamento, che consente di effettuare ricerche efficienti e di mantenere l'ordine dei dati all'interno dell'albero. Un albero binario di ricerca può essere utilizzato per memorizzare e recuperare dati in modo rapido, grazie alla sua struttura gerarchica e alla proprietà di ordinamento.
16. **Come si fanno le operazioni: insert, ricerca di una chiave, max, min, cancellazione di una chiave:**
- A. Insert: Confronta la chiave con i valori dei nodi e inseriscila in un nuovo nodo vuoto nel posto corretto.
 - B. Search: Confronta la chiave con i valori dei nodi e trova la corrispondenza nel percorso giusto dell'albero.
 - C. Max: Procedi verso il sottoalbero destro fino a raggiungere il nodo senza figlio destro, che contiene il valore massimo.
 - D. Min: Procedi verso il sottoalbero sinistro fino a raggiungere il nodo senza figlio sinistro, che contiene il valore minimo.
 - E. Delete: Trova il nodo da cancellare, regola i collegamenti dei nodi circostanti e gestisci i casi di uno o due figli per mantenere la proprietà dell'albero binario di ricerca.
17. **Come si estrae una chiave massima avendo a disposizione un heap?**
- A. Prendi il valore nella radice dello heap, che rappresenta il valore massimo.
 - B. Sostituisci il valore nella radice con l'ultimo valore dell'heap.
 - C. Riduci la dimensione dell'heap di uno.
 - D. Applica la procedura di heapify per ripristinare la proprietà dell'heap nella radice, spostando l'elemento più grande verso la posizione corretta.

18. **Quanto tempo ci si impiega ad estrarre una chiave massima avendo a disposizione un heap?** Il tempo richiesto per estrarre la chiave massima da uno heap dipende dalla sua dimensione, chiamata n . L'operazione di estrazione richiede un tempo di esecuzione di $O(\log n)$ in media, in quanto è necessario eseguire l'operazione di heapify per mantenere la struttura dell'heap corretta. In altri termini, il tempo di esecuzione cresce in modo logaritmico con la dimensione dell'heap.
19. **Dato un nodo interno x di uno heap vettore A dove si trova il figlio sinistro e il figlio destro di x in A , se x ha indice i (assumendo la rappresentazione ad albero di uno heap)?** Nella rappresentazione ad albero di uno heap vettore A , il nodo interno x con indice i avrà il figlio sinistro nell'array A all'indice $2i+1$ e il figlio destro all'indice $2i+2$. Questa relazione si basa sul fatto che gli indici nell'array iniziano da 0. Quindi, se hai un nodo interno x con indice i in uno heap vettore A , puoi trovare il figlio sinistro di x all'indice $2i+1$ e il figlio destro di x all'indice $2i+2$ nell'array A .
20. **Come puoi definire l'altezza di un albero binario?** L'altezza di un albero binario è definita come la lunghezza massima del percorso dalla radice dell'albero a una delle sue foglie. In altre parole, rappresenta il numero massimo di archi che devono essere attraversati per raggiungere la foglia più lontana dall'alto. L'altezza di un albero binario vuoto è considerata 0, mentre l'altezza di un albero con un singolo nodo è 1. L'altezza di un albero binario può fornire informazioni sull'equilibrio e la complessità delle operazioni svolte sull'albero, come l'inserimento, la cancellazione o la ricerca dei nodi.
21. **Quale è l'altezza massima di uno heap di n nodi? Perché?** L'altezza massima di uno heap di n nodi è $\log_2(n+1) - 1$. Questo deriva dalla proprietà delle heap complete, in cui tutti i livelli dell'albero sono riempiti tranne forse l'ultimo, che può essere incompleto. L'altezza massima viene determinata dal numero di livelli dell'albero, che è logaritmico rispetto al numero di nodi.
22. **Contare in un albero binario completo, il numero di nodi interni, le foglie:** In un albero binario completo, il numero di nodi interni è $(\text{nodi_totali} - 1)$ diviso per 2, mentre il numero di foglie è $(\text{nodi_totali} + 1)$ diviso per 2.
23. **Quanto costa nel caso peggiore la ricerca di una chiave in albero binario di ricerca in funzione dei nodi n dell'albero?** Nel caso peggiore, la ricerca di una chiave in un albero binario di ricerca richiede un tempo proporzionale al numero totale di nodi presenti nell'albero. Questo si verifica quando l'albero è completamente sbilanciato, con una struttura lineare in cui ogni nodo ha solo un figlio. In questa situazione, la ricerca deve attraversare tutti i nodi lungo il percorso dalla radice alla chiave desiderata. Pertanto, il tempo di ricerca nel caso peggiore è direttamente correlato al numero di nodi dell'albero.
24. **Lista:** Una lista è una collezione ordinata di elementi, in cui ogni elemento è accessibile tramite un indice univoco. Le liste consentono l'inserimento, la modifica e l'eliminazione di elementi in modo dinamico, consentendo l'aggiunta e la rimozione di elementi in qualsiasi posizione. Possono essere implementate come strutture di dati dinamiche o come array statici.

25. **Coda:** Una coda è una struttura dati che segue il principio del "First-In-First-Out" (FIFO). Gli elementi vengono inseriti nella coda da un'estremità chiamata "rear" e vengono rimossi dalla coda dall'altra estremità chiamata "front". Le operazioni principali su una coda sono l'inserimento di un elemento in coda (enqueue) e la rimozione di un elemento dalla parte anteriore della coda (dequeue).
26. **Pila:** Una pila è una struttura dati che segue il principio del "Last-In-First-Out" (LIFO). Gli elementi vengono inseriti o rimossi solo dall'estremità superiore della pila. L'operazione di inserimento di un elemento in cima alla pila è chiamata "push", mentre l'operazione di rimozione dell'elemento superiore è chiamata "pop". Le pile sono utilizzate per gestire la chiamata di funzioni, la valutazione delle espressioni e altre situazioni in cui è necessario mantenere un ordine di accesso specifico.